

Back To Practice

Q1

Save

First-Come,First-Served CPU Scheduling Algorithm

Completed

Write a C program to Implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : Java (OpenJDK 13.0.1)

```
1 import java.util.Scanner;
2
3 public class FCFS {
4
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int n;
9         System.out.println("Enter number of processes: ");
10        n = sc.nextInt();
11
12        int[] at = new int[n];
13        int[] bt = new int[n];
14        int[] ct = new int[n];
15        int[] tat = new int[n];
16        int[] wt = new int[n];
17
18        for (int i = 0; i < n; i++) {
19            System.out.println("Enter Arrival Time and Burst Time for P" + (i + 1) + ": ");
20            at[i] = sc.nextInt();
21            bt[i] = sc.nextInt();
22        }
23
24        int time = 0;
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Q1

Save

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

Select Language : Java (OpenJDK 13.0.1)

```

25
26 for (int i = 0; i < n; i++) {
27
28     if (time < at[i]) {
29         time = at[i];
30     }
31
32     time += bt[i];
33     ct[i] = time;
34
35     tat[i] = ct[i] - at[i];
36     wt[i] = tat[i] - bt[i];
37 }
38
39 double avgTAT = 0, avgWT = 0;
40
41 for (int i = 0; i < n; i++) {
42     avgTAT += tat[i];
43     avgWT += wt[i];
44 }
45
46 avgTAT /= n;
47 avgWT /= n;
48

```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Q1

Save

First-Come,First-Served CPU Scheduling Algorithm

Completed

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT):** The time at which a process finishes execution.
- **Turnaround Time (TAT):** Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT):** The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.

```

33         ct[i] = time;
34
35         tat[i] = ct[i] - at[i];
36         wt[i] = tat[i] - bt[i];
37     }
38
39     double avgTAT = 0, avgWT = 0;
40
41     for (int i = 0; i < n; i++) {
42         avgTAT += tat[i];
43         avgWT += wt[i];
44     }
45
46     avgTAT /= n;
47     avgWT /= n;
48
49     system.out.printf("\nAverage Turnaround Time: %.2f\n", avgTAT);
50     System.out.printf("Average Waiting Time: %.2f\n", avgWT);
51
52     sc.close();
53 }
54 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

Custom Test

Inputs:

Expected Outputs:

Completed

First-Come,First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come, First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

Completion Time (CT): The time at which a process finishes execution.

Turnaround Time (TAT): Total time taken from arrival to completion.
\$TAT = CT - AT\$

Waiting Time (WT): The time a process spends waiting in the ready queue.
\$WT = TAT - BT\$

Input Format

I. The first input line contains an integer representing the number of processes.

Sample Test Case

Test Case - 1

Input :
3
0 2
1 2
2 3

Expected Output :
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

Output :

Test Case - 2

Input :
3
0 5
5 3
2 2

Expected Output :
Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Back To Practice

Save

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulae:

- \$Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)\$
- \$Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)\$

Input Format

- The first line of input contains an integer representing the number of processes

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i, time = 0, completed = 0;
5
6     printf("Enter number of processes: \n");
7     scanf("%d", &n);
8
9     int at[n], bt[n], ct[n], tat[n], wt[n], visited[n];
10
11     for(i = 0; i < n; i++) {
12         printf("Enter Arrival Time and Burst Time for P%d: \n", i + 1);
13         scanf("%d %d", &at[i], &bt[i]);
14         visited[i] = 0;
15     }
16
17     while(completed < n) {
18         int min = 9999, index = -1;
19
20         for(i = 0; i < n; i++) {
21             if(at[i] <= time && visited[i] == 0) {
22                 if(bt[i] < min) {
23                     min = bt[i];
24                     index = i;
25                 }
26             }
27         }
28
29         time += min;
30         completed++;
31         printf("Process %d completed at time %d\n", index + 1, time);
32     }
33 }
```

Test Cases 2

Run Sample Cases

Run All Cases